

Lesson 13

by Lorraine Gaudio

Lesson generated on November 15, 2025



BOISE STATE UNIVERSITY

Contents

1	☒ Let's Bake a Plot! Layering Data with ggplot2.	2
2	☒ Packages	3
3	What is ggplot2?	4
4	The Blank Canvas	5
5	Mapping Aesthetics	7
6	Adding Geometric Objects (Geoms)	9
6.1	Scatterplot Example	9
6.2	Layering geoms	12
6.3	Histogram Example	13
6.4	Density Plot	16
6.5	Overlaid Histograms	17
6.6	Boxplot	18
7	Labels and Themes	20
8	☒ Assignment	22
8.1	Task 0	22
8.2	Task 1	22
8.3	Task 2	22
8.4	Task 3	23
8.5	Task 4	23
8.6	Task 5	23
8.7	Task 6	24
9	Save and Upload	25
10	Today you practiced:	26

1. ☒ Let's Bake a Plot! Layering Data with ggplot2.

In lesson 12, we did simple data visualizations using R's built-in function `hist()`. Today, we use the package `ggplot2` to turn data frames into clear, publication-quality plots. Specifically, we'll look at the grammar of graphics.

To begin Lesson 13, follow these steps:

1. Open your course project for RStudio
2. Create a new file. From the file types we have used so far, pick which file type you want to use. (File > New File > ???).
3. Type in the code provided in this document as you follow along with the video. Pause the video at anytime to answer assignment questions, dig deeper or add memo notes.

Lesson Overview

By the end of Lesson 13 you will be able to:

1. ☐ Remember – State the steps: `data` ☐ `aes()` ☐ `geom()`.
2. ☐ Understand – Describe how `aes()` maps variables to plot elements.
3. ☐ Apply – Build scatterplots, histograms, and boxplots.
4. ☐ Analyze – Use color or fill aesthetics to compare groups.
5. ☐ Evaluate – Choose suitable geoms and themes for a question.

Keep these goals in mind as you move through each section.

2. ☒ Packages

Install once (if needed): `install.packages("dplyr"); install.packages("dslabs")`

```
install.packages("ggplot2")
```

Load the packages at the start of every session:

```
library(dslabs) # Data science labs package  
library(dplyr)  # Data manipulation package  
library(ggplot2) # Data visualization
```

3. What is ggplot2?

Open the help file for `ggplot()` to see its formal arguments and description.

```
?ggplot # skim the Details section
```

□ The SYNTAX (basic/general):

```
ggplot(data = NULL, mapping = aes(), ..., environment = parent.frame())
```

The three core steps of ggplot2 are: (1) supply a data frame, (2) map aesthetics with `aes()`, and (3) add geometric objects (geoms) with `geom_*()` functions.

4. The Blank Canvas

We'll start building a plot step-by-step. First, we create a blank canvas with `ggplot()`. The data set we'll use is `divorce_margarine` from the `dslabs` package.

```
data("divorce_margarine") # Load the divorce_margarine data
View(divorce_margarine)   # View the data frame
```

□ Goal. Start every plot by piping a data frame into `ggplot()`.

```
df %>% ggplot()
```

```
# □ Code Example: Blank canvas
divorce_margarine %>%
  ggplot()
```

```
# Without piping:
ggplot(data = divorce_margarine)
```

□ Comment: The result is an empty canvas.

R does not throw an error. □ Note that ggplot needs three main steps (data → aes() → geom_*()).

5. Mapping Aesthetics

Review the syntax for ggplot2 again.

- The SYNTAX (basic): `ggplot(data = df, mapping = aes(x, y, other aesthetics))`

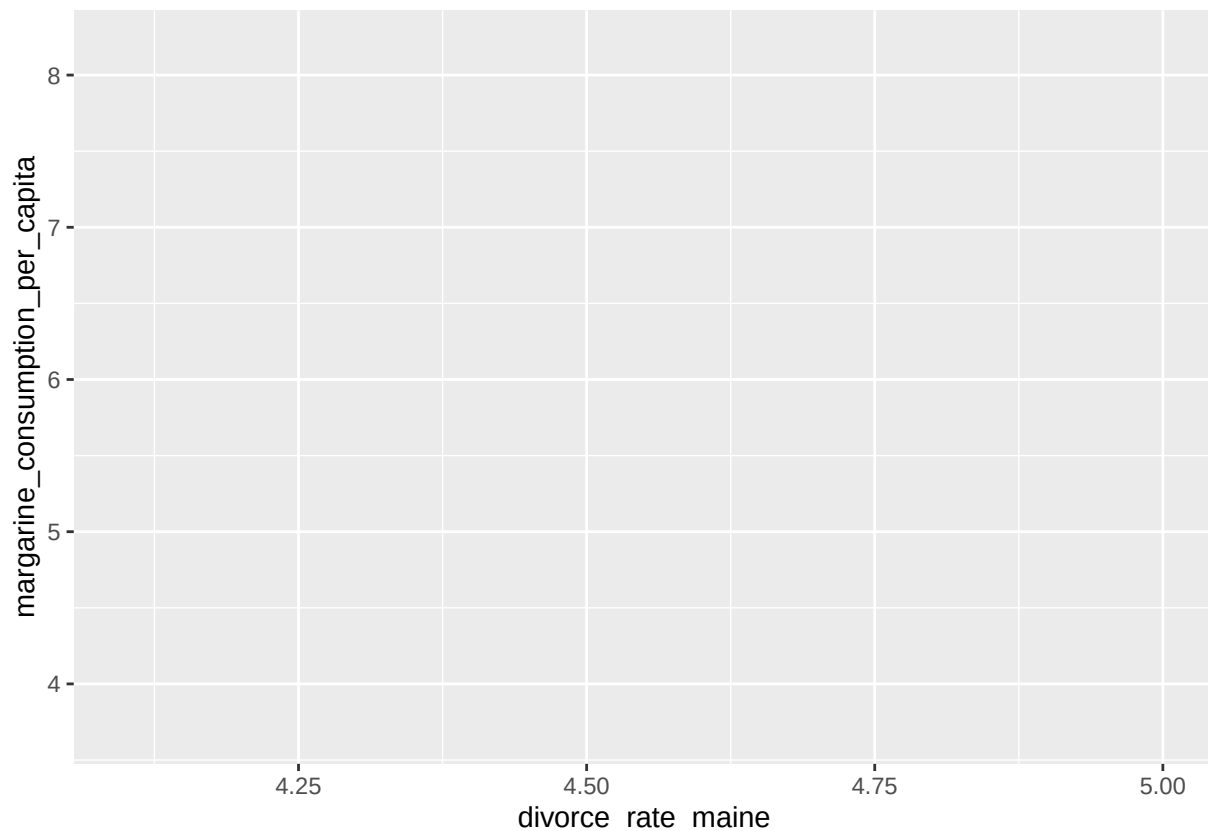
The `aes()` argument supplies mapping information. Mapping refers to connecting your data variables to visual properties (aesthetics) of the plot. It's how you tell ggplot which column in your data should control which aspect of the visualization.

```
?aes
```

- The GOAL: Map divorce rate to the x-axis and margarine consumption to the y-axis using `aes()`.

□ *Code Example: Mapping aesthetics*

```
divorce_margarine %>%  
  ggplot(aes(x = divorce_rate_maine, y = margarine_consumption_per_capita))
```



- Comment: The plot is still blank because we haven't added any geometric objects (geoms) yet. The axes are now labeled by the mapped variables, even if there are no points yet. `aes()` connects

data columns to visual positions. The `aes()` step comes before choosing a geom because we are first defining the mapping (what goes on the x- and y-axes, colors, etc.) and then deciding how to draw those mapped variables (points, lines, bars, etc.).

□ Reflection: The aesthetics describe the underlying structure of the plot, and the geoms are just different ways of rendering that structure. By setting `aes()` inside `ggplot()` we create a shared mapping that all later geoms can reuse, instead of repeating ourselves for each layer.

6. Adding Geometric Objects (Geoms)

Geometric objects (geoms) are the actual shapes drawn on the plot: points, lines, bars, boxplots, etc. Each geom answers the question: *Given these aesthetics, how should the data be drawn?*

6.1 Scatterplot Example

`geom_point()` is a geometric object (or “geom”) in `ggplot2` that creates a scatter plot by placing points at the specified x and y coordinates.

```
?geom_point
```

□ The SYNTAX:

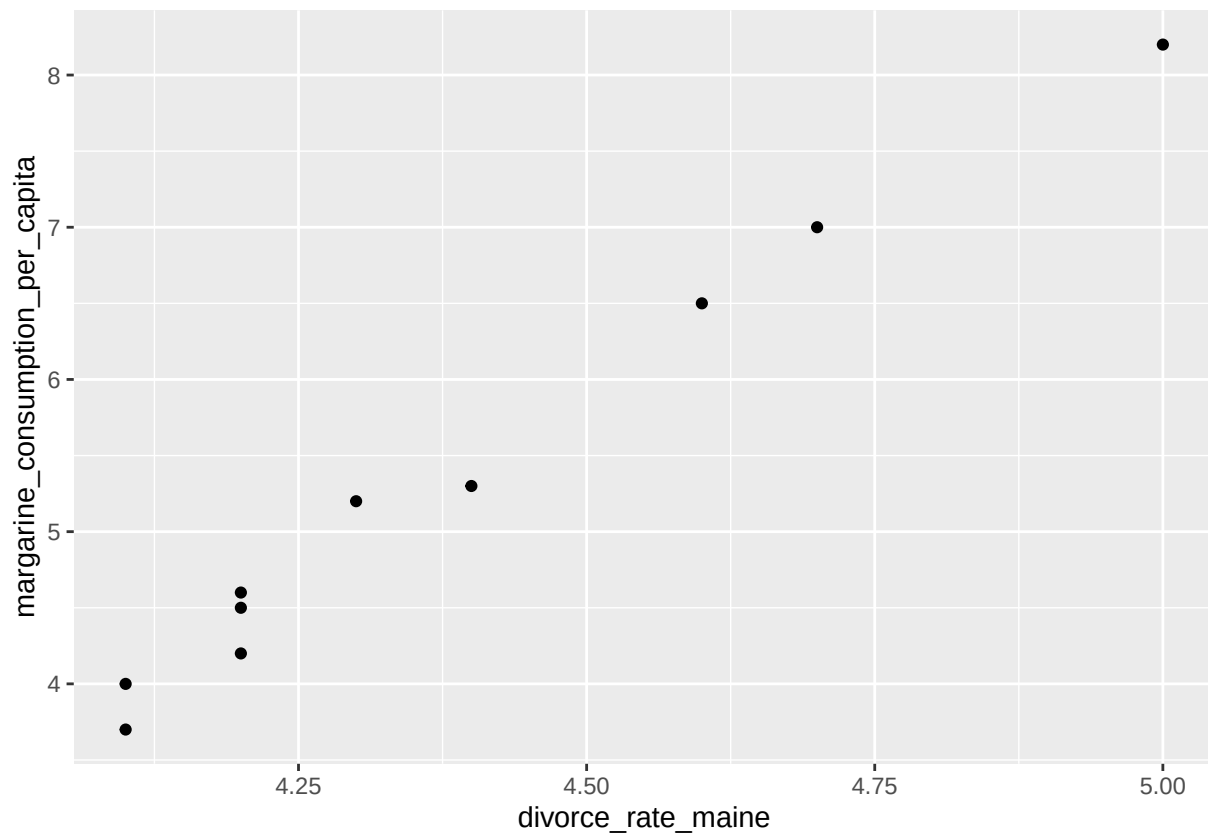
```
geom_point(mapping = NULL, data = NULL, ..., na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
```

□ Notice that all of the arguments are have default settings because `geom_point()` can inherit the data and aesthetic mappings from the initial `ggplot()` call.

□ When using `ggplot2`, geoms are added to the plot using the `+` operator.

1. □ The GOAL: Add points to the plot using `geom_point()`.

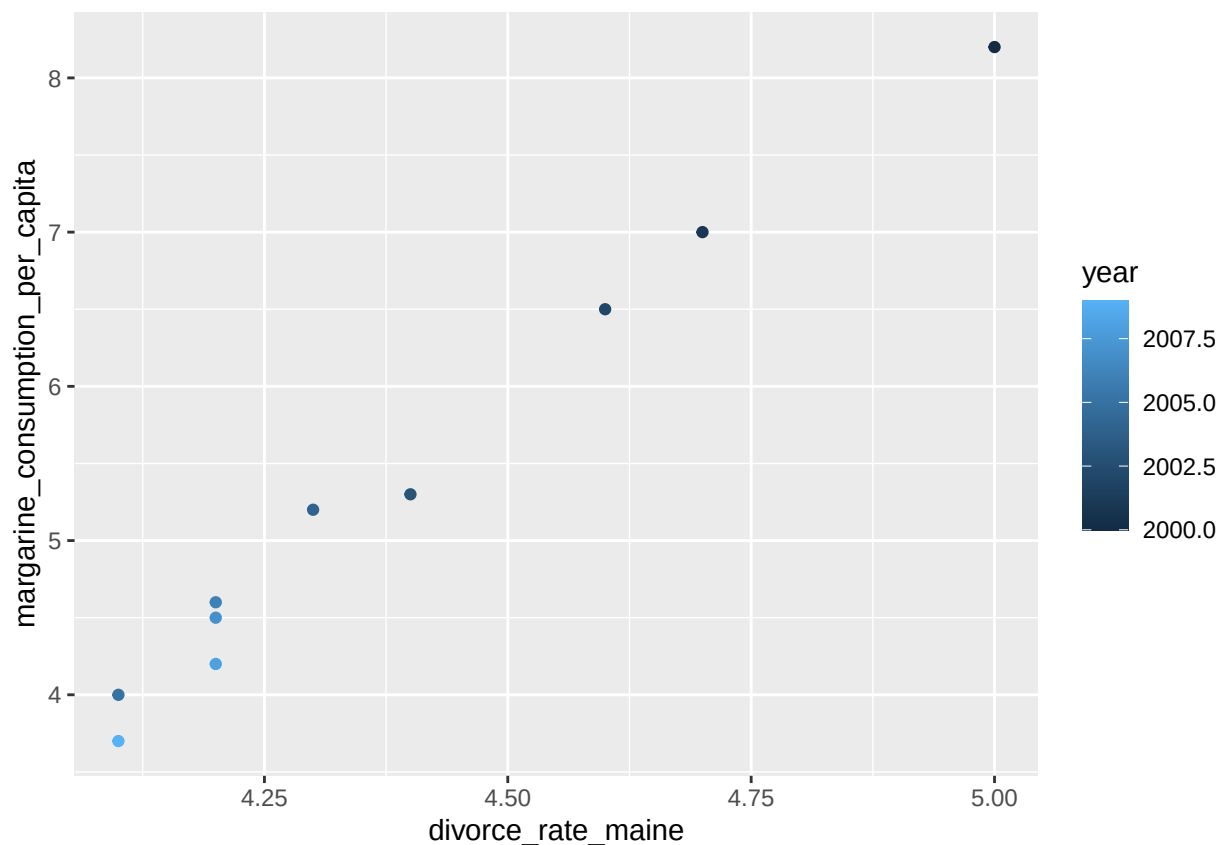
```
# □ Code Example: Scatterplot
divorce_margarine %>%
  ggplot(aes(divorce_rate_maine, margarine_consumption_per_capita)) +
  geom_point()
```



□ Comment: There are points showing a linear, positive association between divorce rate and margarine consumption per capita.

2. □ The GOAL: Map additional aesthetics inside `geom_point()`: color by year, size by divorce rate, and transparency (alpha) by margarine consumption.

```
divorce_margarine %>%
  ggplot(aes(x = divorce_rate_maine, # Global aes mapping: x-axis
             y = margarine_consumption_per_capita)) + # Global aes mapping: y-axis
  geom_point(aes(color = year)) # Local aes mapping: color points by year
```



□ Wait, you might be thinking, “`aes()` is already defined in `ggplot()`, why are we using it again in `geom_point()`?” The `aes()` in `ggplot()` are global mappings for the entire plot and apply to all layers. When we use `aes()` inside a specific geom like `geom_point()`, we’re defining mappings that are specific to that layer. This allows for more granular control over how different layers of the plot behave.

`geom_point()` understands these aesthetics:

- x, y: Position (required)
- alpha: Transparency
- color: Point color
- fill: Fill color (for shapes with fill)
- shape: Point shape (0-25)
- size: Point size
- stroke: Border thickness

□ Explore and Play: Try adding any of the above aesthetics to `geom_point()`. How does it change the plot?

6.2 Layering geoms

Different geoms tell different stories. Let's explore more geoms!

We've seen that `geom_point()` highlights individual data points. `geom_smooth()` adds a smoothed line to show trends in the data.

?geom_smooth

□ The SYNTAX:

```
geom_smooth(mapping = NULL, data = NULL, method = NULL, se = TRUE, ...)
```

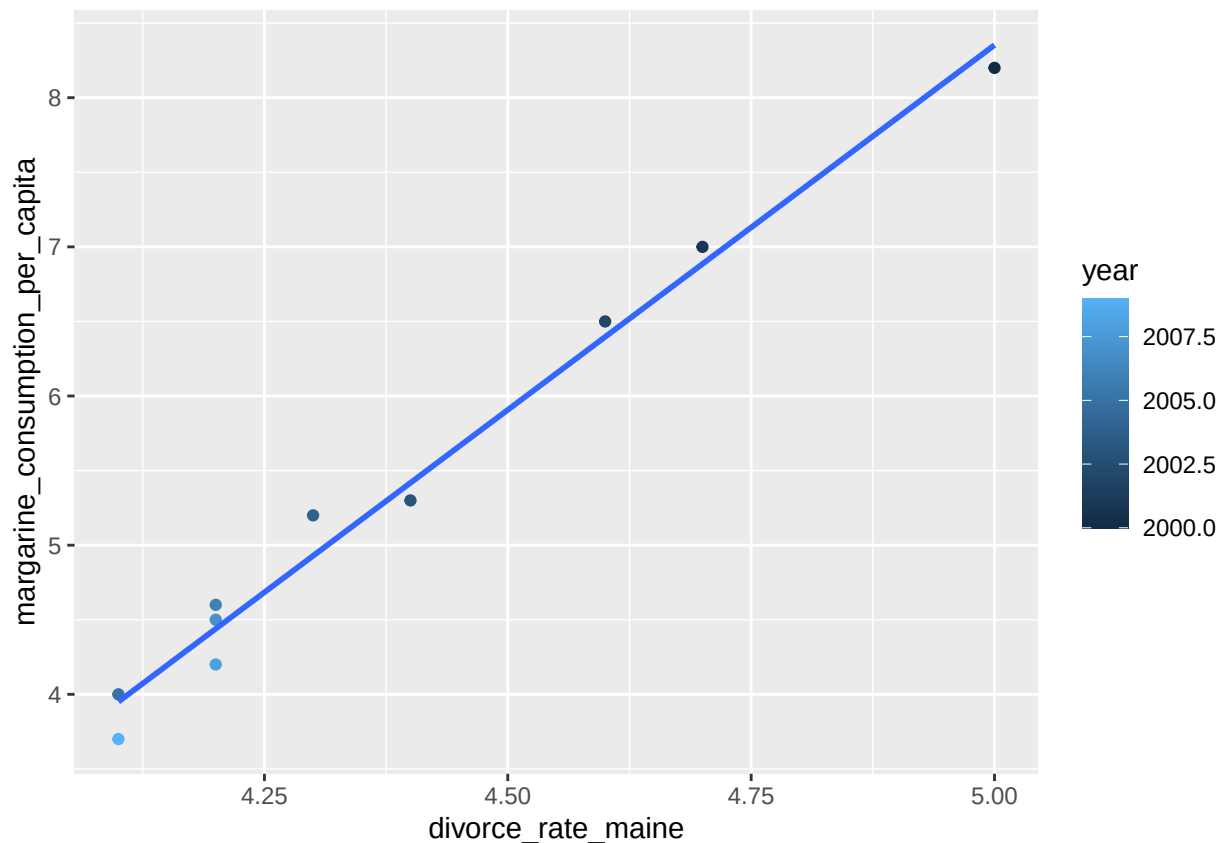
Look at the argument `method`: This controls the smoothing method used. Common options include:

- "lm": Linear model (straight line)
- "loess": Local regression (curved line)

□ The GOAL: Add `geom_smooth()` to the scatterplot.

Since the relationship looks linear, we'll use `method = "lm"` to fit a straight line. We'll also set `se = FALSE` to hide the confidence interval around the line for clarity.

```
# □ Code Example: Scatterplot with trend line
divorce_margarine %>%
  ggplot(aes(x = divorce_rate_maine,
             y = margarine_consumption_per_capita)) +
  geom_point(aes(color = year)) +      # Scatter points
  geom_smooth(method = "lm", se = FALSE) # Add linear trend line
```



□ Comment: The trend line shows a positive linear relationship between divorce rate and margarine consumption.

6.3 Histogram Example

?geom_histogram

□ The SYNTAX:

```
geom_histogram(mapping = NULL, data = NULL, ..., binwidth = NULL, bins = NULL, na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
```

Let's build our own data set to explore histograms.

1. □ The GOAL: Simulate a Tallness data frame with heights for 1,000 people in the USA and Canada each. In the column Country, make two groups labeled "USA" and "CAN". USA heights are centered around 67 inches with a standard deviation of 2.5. CAN heights are centered around 66 inches with a standard deviation of 1.5. Tallness follows a normal distribution.

```
# □ Code
set.seed(2025)
Tallness <- data.frame(
  Country = c(rep("USA", 1000), rep("CAN", 1000)),
```

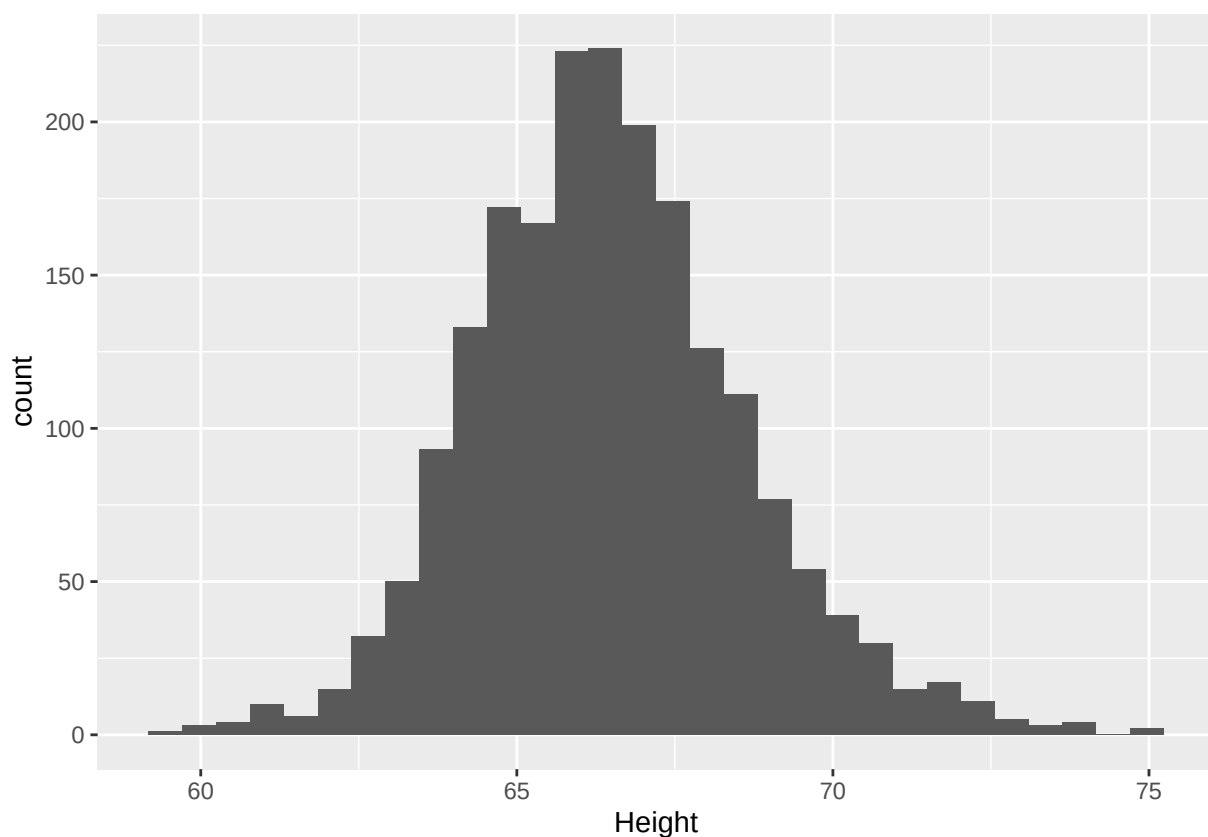
```
Height = c(rnorm(1000, 67, 2.5), rnorm(1000, 66, 1.5))
)
```

```
# □ Verify Tallness data frame
nrow(Tallness)
Tallness %>%
  group_by(Country) %>%
  summarize(n = n(),
    mean_height = mean(Height),
    sd_height = sd(Height),
    .groups = "drop")
```

□ Comment: We have 2,000 rows, 1,000 for each of the two country labels, and heights roughly around expected means and standard deviation.

2. □ The GOAL: Build a histogram of heights using `geom_histogram()`.

```
# □ Code Example: Histogram
Tallness %>%
  ggplot(aes(Height)) + # only x for histograms
  geom_histogram()
```



□ Comment: Histograms visualize the distribution of a single continuous variable. `geom_histogram()` uses the global x aesthetic (`Height`) and so there is no need to repeat the `aes()` inside of the `geom_histogram()` argument. Additionally, the y is computed, not mapped. `geom_histogram()`

uses `stat_bin()` “under the hood” to compute the counts (or density). This is why we do not need to map a column to y. The y-values are produced by the statistic.

`geom_histogram()` understands these aesthetics:

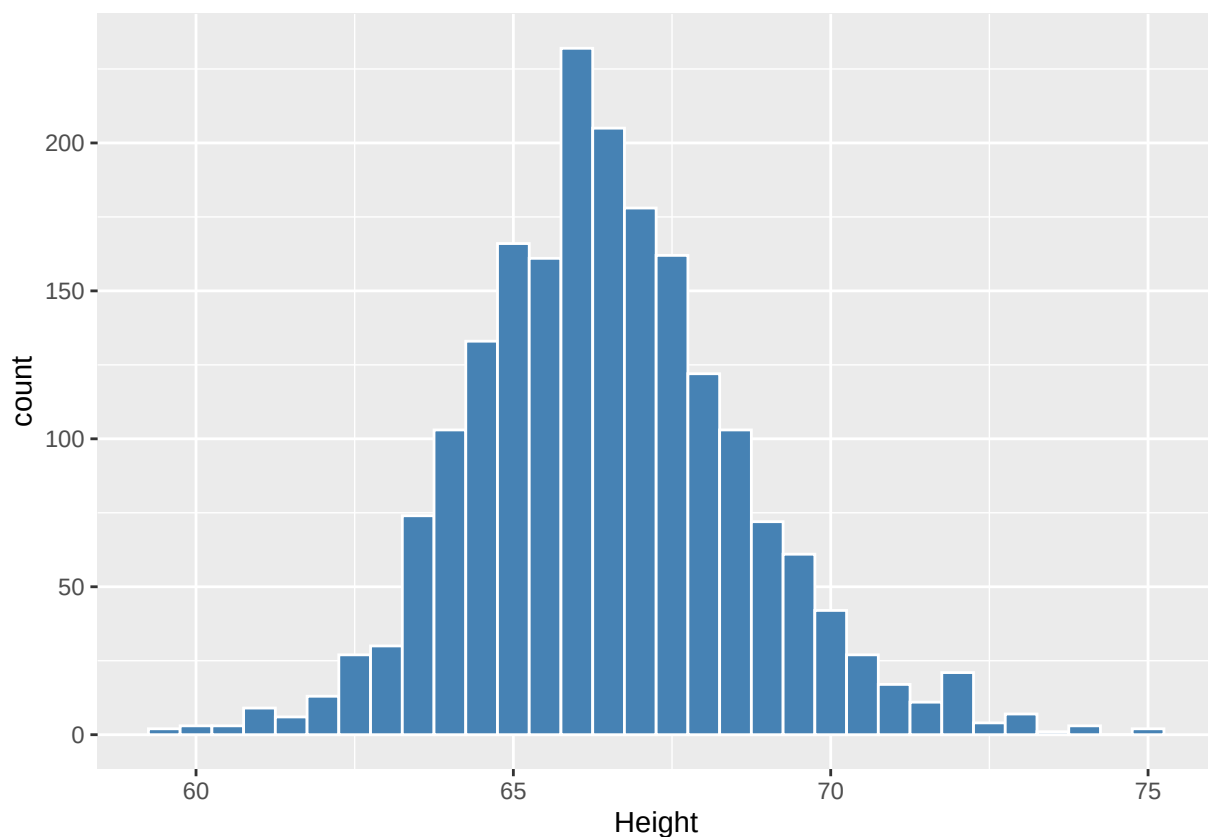
- `x`: Position (required) *and usually entered in `ggplot(aes())`*
- `fill`: Fill color
- `color`: Border color
- `binwidth`: Width of each bin (default is 1)
- `position`: Position adjustment (e.g., “identity”, “dodge”, “fill”)
- `alpha`: Transparency
- `size`: Border thickness
- `weight`: Weight for each observation (optional)
- `linetype`: Line type for borders (optional)

3. ☐ The GOAL: Customize the histogram by setting fill, color, and binwidth.

☐ Code Example: Customized Histogram

Tallness %>%

```
ggplot(aes(Height)) +           # only x for histograms
  geom_histogram(fill = "steelblue",
                 color = "white", # white border gives gap between bars
                 binwidth = 0.5) # Set bin width to 0.5
```



- Comment: The histogram shows a unimodal distribution roughly centered in the high 60s.
- Explore and Play: Change `binwidth = 0.25` inside `geom_histogram()`. Are the overall center and spread the same, even though the “smoothness” changes? Reflect on how binwidth trades off smooth vs. noisy appearance and what you’d choose for a report.
- Break Things! Test out your hand at adding aesthetics `geom_histogram()` understands.

6.4 Density Plot

Now we’ll look at a smooth version of the height distribution using a density curve.

Density plots are smoothed versions of histograms that estimate the probability density function of a continuous variable. They are useful for visualizing the distribution shape without the binning artifacts of histograms.

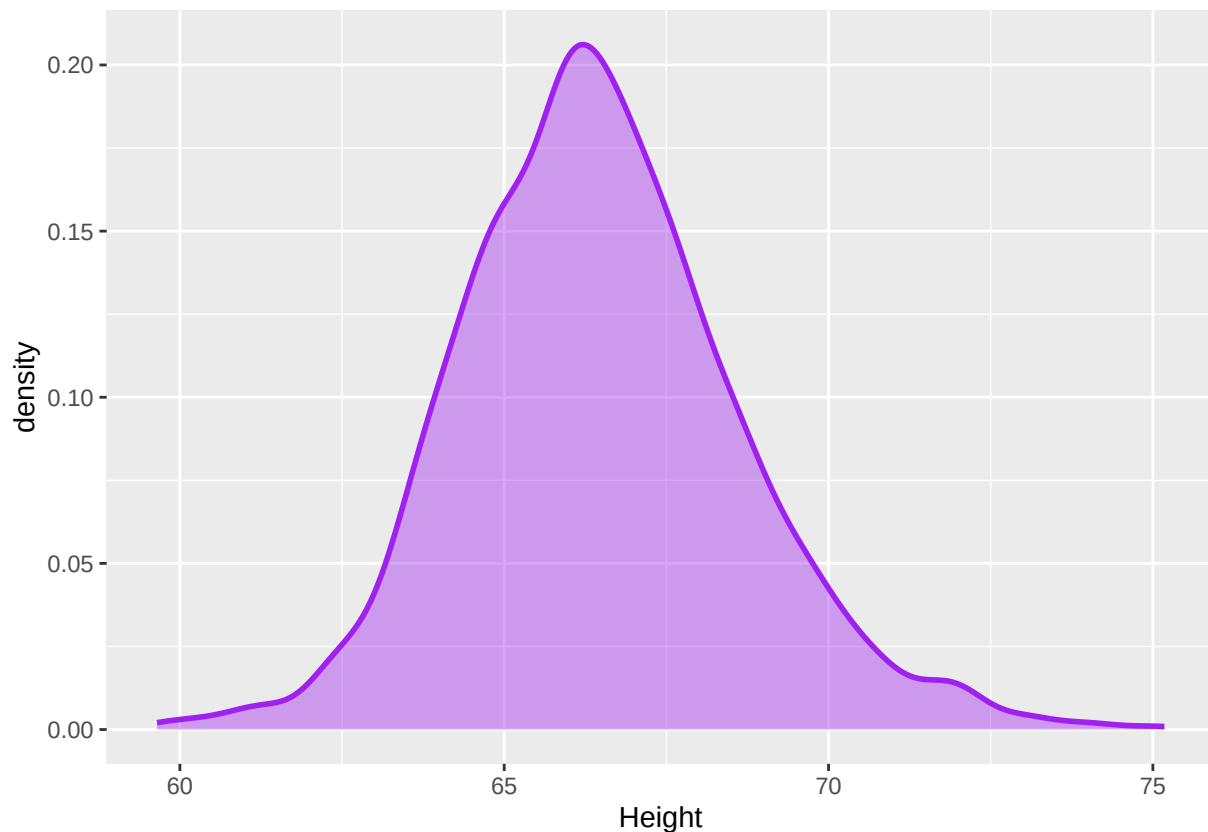
?geom_density

- The SYNTAX:

```
geom_density(mapping = NULL, data = NULL, ..., na.rm = FALSE, show.legend = NA,
inherit.aes = TRUE)
```

- The GOAL: Make a smooth density plot of heights in the Tallness data and emphasize that the vertical axis is density, not counts. Add a density layer that shows a smooth curve for the distribution of heights. Fill the area under the curve, make it partially transparent so the shape is not too heavy.

```
# □ Code Example: Density Plot
Tallness %>%
  ggplot(aes(x = Height)) +
  geom_density(fill = "purple",
    alpha = 0.4,
    color = "purple",
    linewidth = 1)
```



`geom_density()` uses kernel density estimation to create a smooth curve that represents the distribution of the data.

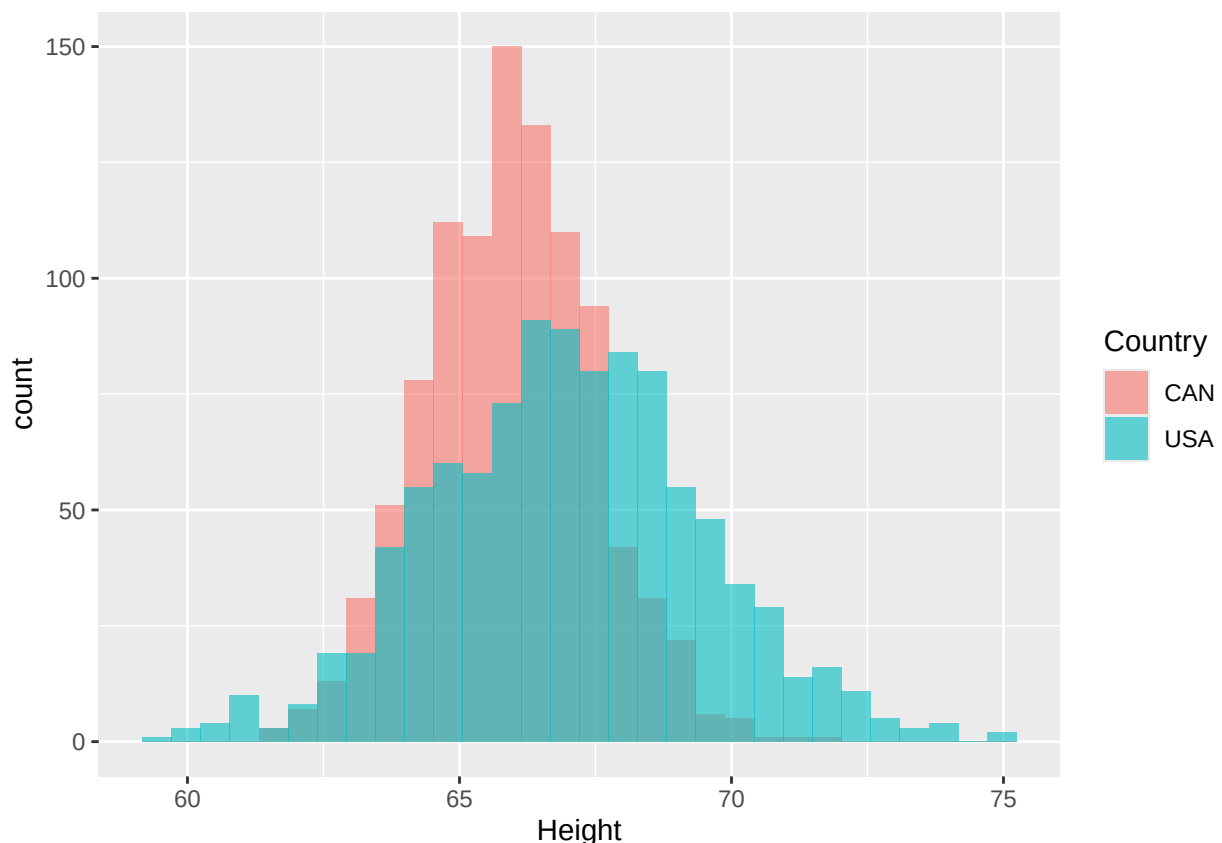
- `x` is the variable whose distribution you want to smooth and is usually entered in `ggplot(aes())`.
- `geom_density()` computes the vertical axis as density by default; the y-values are not mapped from a column.
- `fill` sets the interior color of the density shape.
- `alpha` controls transparency (0 = fully transparent, 1 = solid).
- `color` sets the outline color.
- `linewidth` controls the thickness of the outline.

6.5 Overlaid Histograms

Let's compare the two height distributions by overlaying histograms and using `fill = Country` to distinguish groups.

□ The GOAL: Compare height distributions between USA and CAN by overlaying two histograms using `fill = Country`. Let's store the graph as `tall_compared_fig`. Then, we'll print the figure to take a look.

```
# □ Code Example: Overlaid Histograms
tall_compared_fig <- Tallness %>%
  ggplot(aes(Height, fill = Country)) +
  geom_histogram(position = "identity",
    alpha = 0.6)      # Set transparency to see overlap
# □ Print the figure
print(tall_compared_fig)
```



□ Comment: `fill = Country` in the global `aes()` tells `ggplot()` to make a separate histogram for each country with different fill colors, and automatic a legend. By default, `geom_histogram()` stacks groups. The `position = "identity"` instead puts the bars in the same x-scale so they overlap. The `alpha = 0.6` makes bars transparent (0 = invisible, 1 = solid), so we can see where the two distributions overlap.

6.6 Boxplot

Boxplots summarize distributions using five statistics: minimum, first quartile (Q1), median, third quartile (Q3), and maximum. They are useful for comparing distributions across groups.

```
?geom_boxplot
```

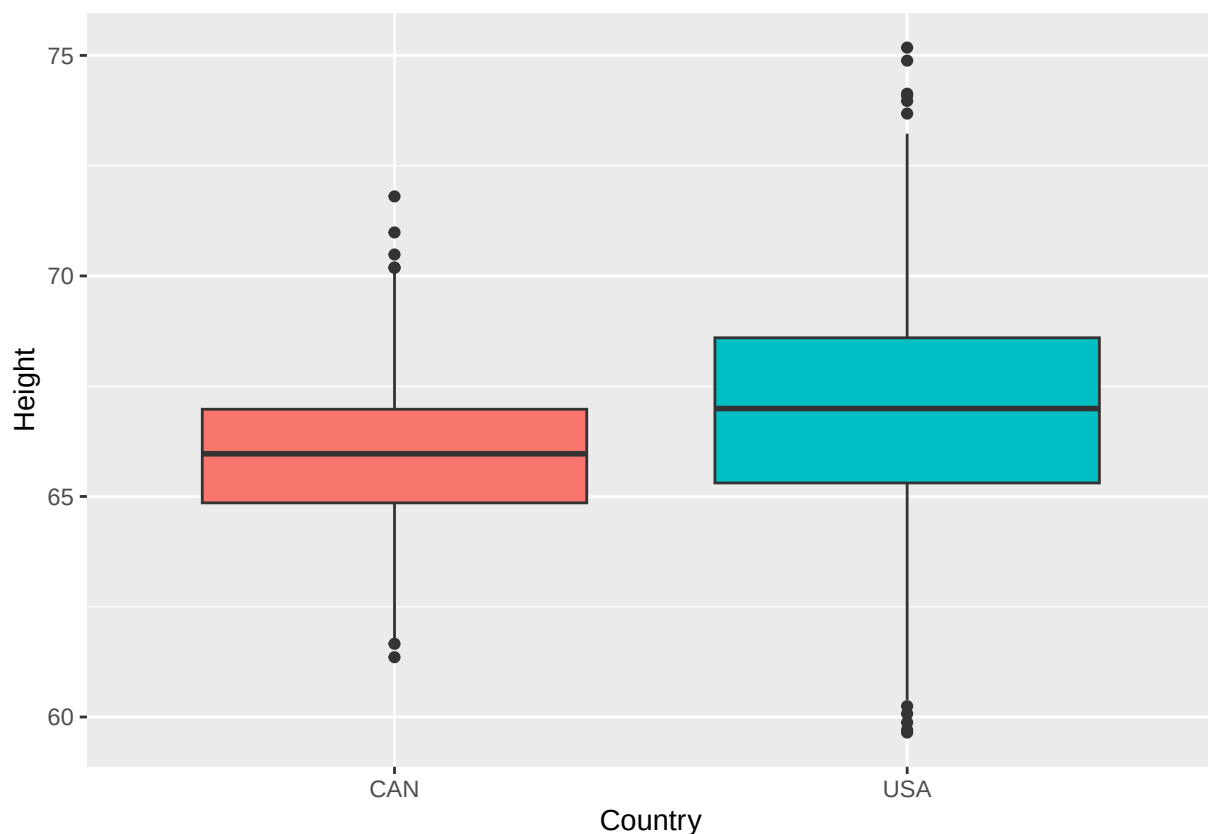
□ The SYNTAX:

```
geom_boxplot(mapping = NULL, data = NULL, ..., outlier.color = NULL, outlier.shape = 19,  
outlier.size = 1.5, na.rm = FALSE, show.legend = NA, inherit.aes = TRUE)
```

We can use boxplots to compare height distributions between USA and CAN.

□ The GOAL: Build side-by-side boxplots of height by country using `geom_boxplot()`. Map Country to the x-axis and Height to the y-axis. Use `fill = Country` to color the boxes by country and set `show.legend = FALSE` to hide the legend. We'll store the boxplot as `Tall_Box`.

```
# □ Code Example: Boxplot  
Tall_Box <- Tallness %>%  
  ggplot(aes(Country, Height, fill = Country)) +  
  geom_boxplot(show.legend = FALSE)  
# □ Print the boxplot  
Tall_Box
```



□ Comment: There are two boxes, one for each country, with USA maybe a bit higher median and wider spread. The countries appear on the x-axis and that each box shows median, IQR, and dots for each of the outliers.

7. Labels and Themes

We can enhance our plots by adding axis labels and applying themes for better aesthetics.

`xlab()` and `ylab()` functions set the labels for the x and y axes, respectively. Simply type the desired label text as a string inside the function.

Additionally, `ggtitle()` adds a title to the plot.

Type `?theme_bw` to explore available themes in `ggtheme {ggplot2}`

□ The GOAL: Add axis labels and a light theme to the boxplot of height by country.

□ *Code Example: Boxplot with labels and theme*

```
Tall_Box <- Tallness %>%
```

```
  ggplot(aes(Country, Height, fill = Country)) +
```

```
  geom_boxplot(show.legend = FALSE) +
```

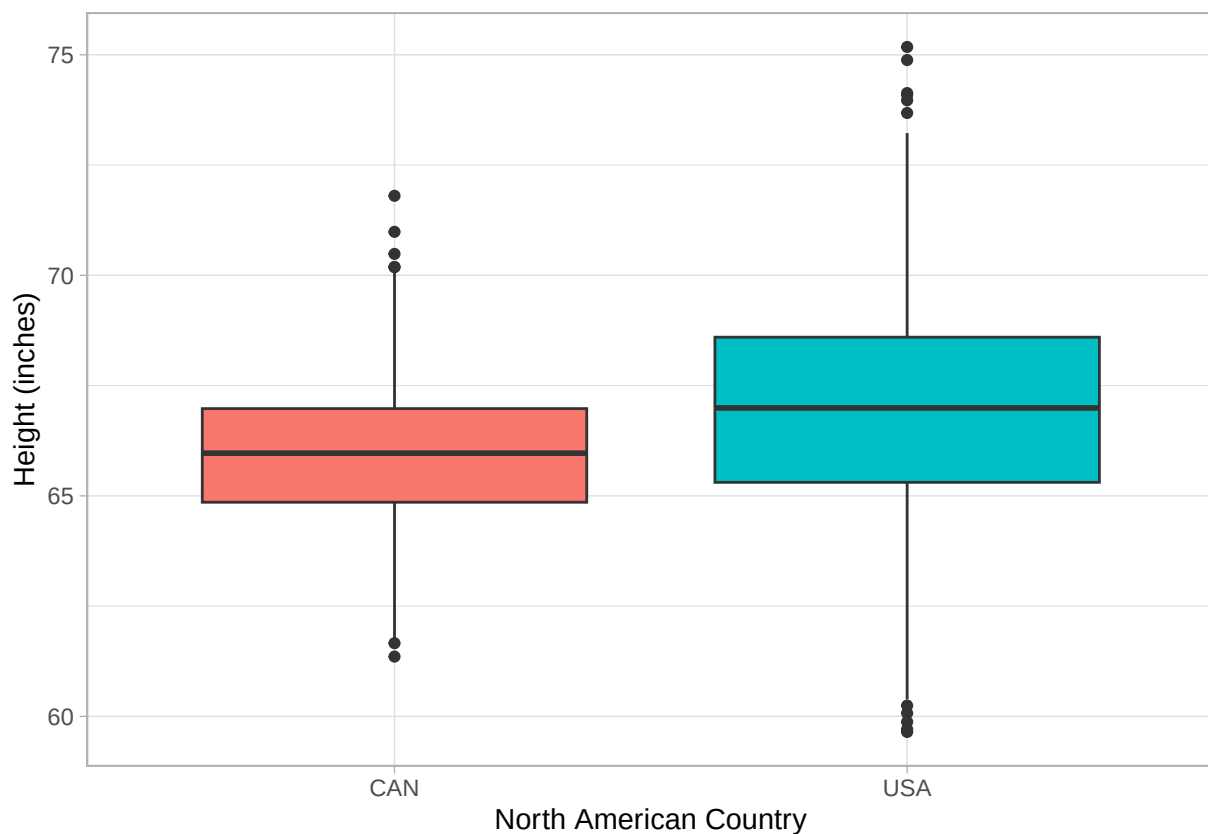
```
  xlab("North American Country") +
```

```
  ylab("Height (inches)") +
```

```
  ggplot2::theme_light()
```

□ *Print the boxplot*

```
Tall_Box
```



□ Comment: `xlab()` and `ylab()` set axis labels. `ggplot2::theme_light()` applies light horizontal grid lines. (The `ggplot2::` prefix explicitly specifies that you're using the `theme_light()` function from the `ggplot2` package. This is useful when multiple packages have functions with the same name.)

8. ✂ Assignment

To earn technical skill points, replace each _____ placeholder (and any TODO comments) with working code or a short written answer. Run each section. When you're done, your workspace should contain FOUR new objects: `Pretty_Flowers`, `Petal_Spread`, `CO2_Graph`, `Tall_Density`. Run each section so that the requested objects appear in your Environment. Save and submit both the "your choice" file with your code (R, Rmd, or Qmd) and the data in your environment (RData). The highest points you can earn in the technical skill practice portion are 3.50.

For every code chunk you add or edit, use the ☐ / ☐ **Goal** → ☐ **Code attempts** → ☐ **Checks (print the figures)** → ☐ / ☐ **Explain the code, the results or what you learned** structure to earn learning skill points. The highest learning skill points you can earn on this submission is 2.50.

8.1 Task 0

☐ Packages

Make sure your script includes code that attached `ggplot2`, `dplyr`, and `dslabs` packages so that your script runs from start to finish without errors.

8.2 Task 1

☐ Quick Recall

List the three core `ggplot2` steps and write a short explanation of each step.

☐ EXPLANATION: "____"

☐ EXPLANATION: "____"

☐ EXPLANATION: "____"

8.3 Task 2

Scatterplot

Create a scatterplot from the built-in iris dataset, and save it in an object called `Pretty_Flowers`. Put sepal length on the horizontal axis and petal length on the vertical axis. Color the points by species so that each species has a different color. Make the points moderately large so they are easy to see. Finally, label the x-axis as "Sepal Length (cm)" and the y-axis as "Petal Length (cm)".

```
# □ Code  
_____ <- _____
```

8.4 Task 3

Boxplot: Group Comparison

Create a boxplot from the built-in iris dataset and store it in an object called `Petal_Spread`. Put species on the horizontal axis and petal width on the vertical axis. For each species, draw a separate boxplot showing the spread and typical values of petal width. Fill each box with a different color according to species, but remove the legend so only the colored boxplots and axis labels indicate which species is which.

```
# □ Code  
_____ <- _____
```

8.5 Task 4

Density plot: split + transparency

Create a density plot using the built-in `CO2` dataset and store the plot in an object called `CO2_Graph`. Put `CO2` uptake on the horizontal axis. Use plant type to define separate smooth curves, so each plant type has its own density curve filled with a different color. Make the filled areas partially transparent so the curves can overlap and still be visible where they intersect. Finally, add the title “CO2 Uptake by Plant Type” (use `ggtitle()`) at the top of the plot.

```
# □ Code  
_____ <- _____
```

8.6 Task 5

□ Histogram vs. Density

Create a graph from the heights dataset and save it in an object called `Tall_Density`. Put height (in inches) on the horizontal axis. Draw a histogram of heights using bars that are 1 inch wide, filled in blue with white borders. Scale the histogram so that the vertical axis shows density (proportion), not raw counts. On top of this, add a smooth density curve that shows the overall shape of the height distribution, using a dark orange line. Fill the area under that density curve with the same dark orange color, but make it partially transparent so you can still see the blue histogram bars underneath. Label the x-axis “Height (inches)” and the y-axis “Density” (**Hint:** See Layering geoms and use `geom_histogram()` first, then `geom_density()`.)

```
# □ Code  
_____ <- _____
```


8.7 Task 6

☐ Reflect:

☐ Write a short paragraph reflecting on when would a density curve be more useful than a histogram?

☐ EXPLANATION: “___”

9. Save and Upload

1. You will be submitting **both** the “your choice” code file (R, Rmd, Qmd) and the workspace file (RData).

10. Today you practiced:

- Created a blank canvas with `ggplot()`.
- Mapped variables inside `aes()`.
- Added geoms for scatter, histogram, and boxplot.
- Used `color` / `fill` to compare groups.
- Applied labels and a light theme for clarity.
- Saved plots as objects for submission.